

Relatório Acadêmico – Estruturas de Dados: Pilha e Fila

1. Informações gerais

Disciplina: Estruturas de Dados

Tema: Implementação de pilha e fila em Java para resolução de problemas cotidianos

Projeto: Banco/ATM e atendimento ao cliente

Membros:

- Carlos Henrique Viana Marinho
- Rafael Berton Martins
- Rafael Padilha de Lima Mattioli

2. Contextualização do problema

O projeto foi desenvolvido para demonstrar duas estruturas fundamentais de dados: pilha e fila, aplicadas em cenários reais de uso cotidiano.

A pilha foi utilizada para representar o estoque de cédulas do caixa eletrônico (ATM). O sistema precisa armazenar notas, retirar a última adicionada durante o saque e verificar se o conjunto está vazio ou ocupado. Já a fila foi empregada para organizar o atendimento de clientes em uma fila de espera, respeitando a ordem de chegada.

Essa abordagem é adequada ao estudo de algoritmos e estruturas de dados porque evidencia o comportamento e a lógica de funcionamento de cada tipo de estrutura, além de permitir a compreensão de operações como criação, inserção, exclusão e busca.

3. Implementação da pilha

A pilha foi modelada no projeto por meio da classe `CashStock`, que representa o conjunto de cédulas de um mesmo valor, e pela estrutura `Stack` da linguagem Java.

3.1 Trecho do código

```
package br.com.cmarinho.models;

import lombok.AllArgsConstructor;
import lombok.Data;

import java.util.Stack;

@Data
@AllArgsConstructor
public class CashStock {
    private final CashEnum typeOfCash;
```

```

private final Stack<Cash> cashStock;

public CashStock(CashEnum cashEnum) {
    this.cashStock = new Stack<>();
    this.typeOfCash = cashEnum;
}

public static CashStock fromNewCashes(int quant, CashEnum cashEnum) {
    var tmpStock = new CashStock(cashEnum);
    for (int i = 0; i < quant; i++)
        tmpStock.cashStock.push(new Cash(cashEnum));
    return tmpStock;
}

public String status() {
    return cashStock.empty() ?
        "empty" : "occupied";
}
}

```

3.2 Funcionamento

A pilha foi criada em Java usando a classe `Stack`, que segue o princípio LIFO (Last In, First Out), ou seja, o último elemento inserido é o primeiro a sair.

Os principais métodos observados:

- `push(...)` : inclusão de um novo elemento na pilha;
- `pop()` : remoção do elemento do topo da pilha;
- `empty()` : verificação de pilha vazia;
- `status()` : retorna se a pilha está vazia ou ocupada;

No contexto do ATM, a cédula de maior valor dentro do estoque representa o último item empilhado; ao sacar dinheiro, a aplicação remove a cédula no topo da pilha, isso torna a operação coerente com a estrutura LIFO.

3.3 Busca e status da pilha

A busca de elementos em uma pilha pode ser feita por meio de iteração ou pelo método `search()`, presente em `java.util.Stack`. No projeto, o status da pilha foi verificado pela função `status()`, que verifica se o conteúdo está vazio. O status também é exibido de forma organizada na classe `ATM`.

Trecho da classe `ATM`:

```

public void printStatus() {
    StringBuilder str = new StringBuilder("\n* ATM STATUS\n"
        + "-----\n");

    StringBuilder values = new StringBuilder();
    StringBuilder statuses = new StringBuilder();
    StringBuilder quantities = new StringBuilder();

    for (CashStock stock : stocks.values()) {
        values.append("%-20s | ".formatted("%d".formatted(stock.getTypeOfCash().value())));
        statuses.append("%-20s | ".formatted("status=" + stock.status()));
        quantities.append("%-20s | ".formatted("quant=" + stock.getCashStock().size()));
    }

    str.append(values)

```

```

        .append("\n")
        .append(statuses)
        .append("\n")
        .append(quantities)
        .append("\n\n");
    System.out.println(str);
}

```

Esse método imprime, em coluna, o valor da nota, seu status (`occupied` ou `empty`) e a quantidade disponível. Isso facilita a visualização do estado do ATM em tempo real.

4. Implementação da fila

A fila foi representada pela classe `ClientSupportQueue`, sendo aplicada ao atendimento de clientes em uma situação cotidiana: organização da fila de espera em um banco ou atendimento de suporte.

4.1 Trecho do código

```

package br.com.cmarinho.models;

import java.util.ArrayDeque;
import java.util.Queue;

public class ClientSupportQueue {
    private final Queue<Client> clients;

    public ClientSupportQueue(int maxSize) {
        this.clients = new ArrayDeque<>(maxSize);
    }

    public boolean addClient(Client client) {
        if (client == null) {
            System.out.println("Cliente nulo não pode ser adicionado.");
            return false;
        }

        if (clients.size() >= getCapacity()) {
            System.out.println("Fila cheia. Cliente não adicionado: " + client.getName());
            return false;
        }

        clients.offer(client);
        System.out.println("Cliente adicionado: " + client.getName() + " | fila: " + clients.size());
        return true;
    }

    public Client attendNextClient() {
        if (clients.isEmpty()) {
            System.out.println("Fila vazia. Nenhum cliente para atender.");
            return null;
        }

        Client attended = clients.poll();
        System.out.println("Atendendo cliente: " + attended.getName());
        return attended;
    }

    public void printStatus(String title) {
        System.out.println("-----");
        System.out.println("* " + title);
        System.out.println("tamanho: " + clients.size());

        if (clients.isEmpty()) {
            System.out.println("fila: vazia");
        } else {
            StringBuilder queueStatus = new StringBuilder("fila: ");
            for (Client client : clients) {

```

```

        queueStatus.append(client.getName()).append(" | ");
    }
    System.out.println(queueStatus);
}

    System.out.println("-----\n");
}
}

```

4.2 Funcionamento

A fila trabalha com o princípio FIFO (First In, First Out), ou seja, o primeiro elemento a entrar é o primeiro a sair.

Os principais métodos utilizados foram:

- `offer(...)` : inclusão de cliente em fila;
- `poll()` : remoção do cliente que está na frente da fila;
- `size()` : verificação do número de elementos na fila;
- `isEmpty()` : valida se a fila está vazia;

Essa estrutura é apropriada para casos como:

- fila de atendimento bancário;
- fila de chamadas de suporte;
- fila de pedidos em um sistema de atendimento.

4.3 Busca e status da fila

A fila não possui busca direta tão direta quanto uma lista, mas é possível localizar um cliente percorrendo os elementos da fila. No código implementado, a busca é demonstrada pela inspeção do conteúdo da fila dentro do `printStats()`, que mostra todos os nomes ordenados pela sequência atual de atendimento.

A visualização do status permite identificar claramente:

- tamanho da fila;
- clientes presentes;
- se a fila está vazia;
- próxima pessoa a ser atendida.

5. Demonstração no método main

A execução principal está organizada para chamar as simulações das duas estruturas em sequência.

```

public static void main(String[] args) {
    simulateClientQueue();
    simulateATM();
}

```

Esse desenho deixa o programa mais simples e organizado, separando a lógica de cada estrutura em métodos específicos. A fila de clientes é simulada em ``simulateClientQueue()`` e a pilha do ATM em ``simulateATM()``.

6. Conclusão

A implementação demonstrou com sucesso as operações fundamentais de pilha e fila em Java, aplicadas em problemas cotidianos: estoque de cédulas em ATM e atendimento de clientes em fila.

A pilha foi utilizada para representar o controle de cédulas, com inserção, remoção e verificação de status. A fila foi usada para organizar a lista de espera, preservando a ordem de chegada e o atendimento correto.

A partir da análise do código, conclui-se que o uso dessas estruturas é essencial para resolver problemas em que a ordem de processamento e o controle de elementos são importantes. A aplicação prática desses conceitos reforça o entendimento teórico e sua utilidade em sistemas reais.

7. Referências do projeto

Classes principais utilizadas:

``CashStock``

``ATM``

``ClientSupportQueue``

``Client``

``Main``

8. Observação

O relatório foi elaborado com foco na implementação e no funcionamento prático das estruturas de dados solicitadas, atendendo ao contexto acadêmico do projeto e às exigências de demonstração de criação, inclusão, exclusão, busca e status da pilha e da fila.